# LEGO EV3 Robot Project Report
# Rubik's Cube Solving Machine


**Department of Mechanical and Mechatronics Engineering**

**A Report Prepared For:**
The University of Waterloo

**Prepared By:**
Siyuan Pan 21128571
Cean Liu 21125520
Leo Zhang 21127977
Aston Ng 21134915
150 University Ave W.
Waterloo, Ontario, N2N 2N2

December 3, 2024

# Table of Contents

# List of Figures

# List of Tables

# Acknowledgements

We would like to express our sincere gratitude to the **University of Waterloo** and the Department of Mechanical and Mechatronics Engineering for providing the resources and support necessary for the successful completion of this project. The facilities and tools available, including the LEGO EV3 kits, were instrumental in turning our vision into reality.

Special thanks are due to our course instructors and TAs for their guidance, feedback, and encouragement throughout the course of the project. Their expertise and constructive criticism helped us refine our designs and overcome technical challenges.

We also extend our appreciation to the creators and maintainers of external resources and tools utilized during the project, such as **LEGO Mindstorms EV3 Education Edition MindCub3r [1]** for insights into the mechanical design, the **Online Rubik's Cube Solver [2]**, and the open-source algorithms hosted on **GitHub [3]**. These resources provided a solid foundation and valuable insights that significantly influenced our approach to solving problems.

Finally, we thank our teammates for their dedication, collaboration, and relentless effort to bring this project to fruition. It is through our collective contributions and shared commitment to excellence that this endeavor was made possible.

# Summary

This report covers the development of a LEGO EV3-based robot designed to solve a 3x3 Rubik's Cube. The objective was to create a fully automated, user-friendly machine capable of both scrambling and solving the cube, making it accessible and enjoyable for beginners. The project integrated mechanical design, software development, and problem-solving into a cohesive system.

The robot's design prioritized stability, precision, and ease of use, featuring a sturdy chassis, a motorized gripping arm, and a rotating base for controlled movements. Ultrasonic sensors and motor encoders were used to process the cube's configuration. Due to the EV3 color sensor's limitations, a creative solution was implemented: scrambling and solving sequences were managed externally using file-based algorithms.

The software was modular, simplifying debugging and future upgrades. Two algorithms were tested: IDA* for efficiency and a Selenium-based web solution for consistency. While IDA* was too slow for EV3 hardware, the Selenium method delivered reliable results despite requiring more moves.

After extensive testing, the robot successfully scrambled and solved the cube, displaying solving times. Despite challenges like sensor inaccuracy and motor alignment, practical solutions resulted in a functional and dependable robot.

Improvements could include upgrading to better sensors or vision-based systems, refining mechanical components for precision, and optimizing algorithms for faster solving. A more user-friendly interface with real-time feedback and full automation would enhance the user experience.

This project demonstrates how mechanical design, software, and creativity can solve real-world problems. The team's innovative approach led to a functional robot with significant potential for future advancements.

# 1.0 Introduction

This report, titled "**LEGO EV3 Robot Project Report: Rubik's Cube Solving Machine**", serves as Team 16's final documentation for the MTE100/MTE121 project. It provides a comprehensive overview of the project, including mechanical and software designs, implementation, testing procedures, revisions, conclusions and recommendations. The focus of the report is on design processes and their alignment with defined constraints and criteria. Detailed calculations such as mechanical stress analysis or advanced explanations of the software code are excluded but referenced where applicable.

## 1.1 Background

The project was assigned on October 11, 2024, as part of the MTE100 and MTE121 courses. Teams were tasked with constructing a functional robot using LEGO EV3 components and sensors. The provided materials included a standard EV3 robot and an additional parts kit, enabling substantial mechanical redesign. The primary objective was to address specific constraints and performance criteria, detailed in Section 3.0.

## 1.2 Problem Identification

Rubik's cubes pose significant challenges for beginners, particularly in restoring the cube to its solved state for systematic practice. This often leads to frustration and inefficiency. To address this issue, Team 16 designed an **automatic Rubik's Cube solving machine**. The robot's purpose is to accept a 3x3 cube in any configuration and return it to a solved state, offering a reliable and user-friendly solution.

To enhance its functionality and engagement, the robot was conceptualized with additional features, such as:

- **Scrambling** the cube to randomized configurations, useful for competitions.
- **Timing** the cube-solving process to foster user engagement and competitive interaction.
- (Optional) Providing **output steps** for solving the cube and creating specific patterns (e.g., a smiley face) for entertainment.

Due to time and resource constraints, not all envisioned features were implemented in the final design, as outlined in Section 2.0.

# 2.0 Scope

The project's scope is defined by the robot's key functionalities, summarized in **Table 1**.

*Table 1 1: Main Functionalities of the Robot*

| Task # | Function |
|---|---|
| 1 | Detects the cube's placement using an ultrasonic sensor and begins operation. |
| 2 | Scrambles the cube into random configurations upon pressing the "Left" or "Right" button. |
| 3 | Solves the cube to achieve six uniform faces upon pressing the "Enter" button. |
| 4 | Outputs the time taken for the operation to the console and shuts down the system. |

The robot's control interface features four primary buttons:

- **Left/Right**: Initiates scrambling.
- **Enter**: Begins solving the cube.
- **Down**: Stops all actions and exits the program (available for press only after one action is fully completed)

## 2.1 Functionality Overview

The robot operates through the following mechanisms:

- **Sensors**:
  - An **ultrasonic sensor** detects the cube's placement.
  - A **color sensor** scans and stores the cube's color configuration in 2D arrays, enabling the robot to determine its next action.
- **Software Logic**: The software monitors the cube's state, utilizing external algorithms to determine solving steps. These steps are stored in and read from external files.
- **Files**: The robot reads from two main external files: the "scrambling.txt" which lists randomly generated steps to scramble the cube and the "output.txt" which lists steps generated from external algorithms to solving the cube.
- **Actuators**:
  - A motorized **arm** flips the cube counterclockwise.
  - A motorized **base** rotates the cube, allowing for controlled layer movements.
  - A motorized **colour sensor** that moves up and down, forward and backward, allowing for measurement of the cube's colour configuration.
- **Shut Down Procedure**: Tasks are completed either upon executing all programmed steps or by user interruption via the "Down" button.

### 2.2 Scope Adjustments

Several adjustments were made during the project to streamline scope:

1. **Removed Special Patterns**: The team opted to exclude features like specific cube patterns (e.g., crosses) due to limited time and technical complexity.
2. **Excluded Step Output**: Displaying solving steps was omitted to reduce project complexity. Users can observe these steps directly during the robot's operation, making the feature redundant.

These changes reflect the team's prioritization of functionality, usability, and efficient time and resource allocation.

## 3.0 Constraints and Criteria

The design and implementation of the Rubik's Cube-solving robot were guided by a set of constraints and criteria that defined the project's scope and performance expectations. The constraints established the mandatory technical requirements, while the criteria served as benchmarks to evaluate the robot's success in meeting the project goals.

### 3.1 Constraints

The constraints imposed on the project were as follows:

1. **Safety Compliance:** The robot had to meet all relevant safety standards to ensure secure and reliable operation under all conditions.
2. **Mechanical Redesign**: The project required a significant mechanical redesign of the standard EV3 robot configuration, utilizing the provided LEGO EV3 kit and additional parts.
3. **Motor Usage**: The robot was limited to using three motors, each with controlled and precise movement to execute necessary tasks. Additional motors can be requested; however, it is in limited amount.
4. **Input Diversity:** Four distinct types of inputs were mandated for use in the robot and the team has chosen the following to implement in the robot:
    - **Ultrasonic Sensor:** Used to detect the presence of the Rubik's Cube and ensure proper placement on the base.
    - **Color Sensor:** Responsible for reading and analyzing the colors on the cube's faces.
    - **Motor Encoders:** Ensured precision in motor movements during cube manipulation.

- **EV3 Buttons:** Acted as the robot's user interface, controlling various processes such as scrambling (via Left/Right buttons), solving (via the Enter button), and shutting down the system (via the Down button).

5. **RobotC File I/O:** The robot's software had to incorporate file input and output to read instructions for solving and scrambling the cube, as well as generate necessary operational data.
6. **Timers:** The robot was required to display the time taken to complete the cube-solving process on the console.
7. **Software Requirements:** The software had to include essential programming constructs such as loops and decision-making structures (if statements). Additionally, the program was required to have at least six non-trivial functions, excluding the main function, with:
    - At least one function returning a value.
    - At least one function accepting parameters.

These constraints defined the technical boundaries within which the team had to operate, ensuring that both hardware and software components were integrated effectively while maintaining feasibility.

## 3.2 Criteria

The project criteria were established to evaluate the robot's functionality, reliability, and usability. The key criteria included:

1. **Cube Solving:** The robot had to accept a 3x3 Rubik's Cube in any scrambled configuration and solve it to achieve six uniform faces.
2. **Random Scrambling:** The robot needed to scramble the cube into a random configuration to support competition and practice scenarios.
3. **Stability and Precision**: During operation, the robot was required to manipulate the cube stably and precisely, ensuring that the cube remained securely positioned and that movements were smooth and accurate.
4. **Error Handling:** The robot was expected to manage unexpected situations effectively, demonstrating robustness and adaptability during operation.

These criteria ensured that the robot's design addressed both functional and user-centric objectives, emphasizing reliability, ease of use, and operational stability.

## 3.3 Impact of Constraints and Criteria

The constraints and criteria played a pivotal role in shaping the design and development of the robot.

The requirement to integrate four distinct input types had a significant influence on both the mechanical and software designs. Sensors such as the ultrasonic and color sensors dictated the robot's ability to detect and interpret the state of the cube, while the EV3 buttons provided an intuitive interface for user interaction. This constraint also necessitated a well-structured software design to accommodate these inputs.

The primary criterion of solving the cube was central to the project and guided key mechanical design decisions. To manipulate the cube effectively, the team developed a motorized arm capable of flipping the cube and a rotating base mechanism for layer manipulation. These components ensured that the robot could perform the necessary movements to solve the cube reliably. The dimensions and layout of the robot were dictated by the size of the 3x3 cube, ensuring compatibility and efficient operation.

Stability and precision were also critical design considerations. The team ensured that the motorized arm applied appropriate force and moved at controlled speeds to prevent misalignment or damage to the cube. Stabilization techniques, such as securely anchoring the cube during operations, were implemented to enhance reliability.

While some constraints, such as the inclusion of loops and decision-making structures, were essential for software functionality, they were less influential as they are standard in most programming tasks. Similarly, the criterion for handling unexpected situations, although important, was difficult to address early in development due to the unpredictability of potential issues.

Initially, the team aimed to optimize the robot's efficiency in solving the cube. However, achieving maximum efficiency would have required six motors to control independent arms, exceeding the available resources. Instead, the team prioritized reliable functionality over efficiency, ensuring that the robot could consistently solve the cube within the constraints of the provided hardware.

# 4.0 Mechanical Design and Implementation

## 4.1 Overall Design

The mechanical design of the robotic Rubik's Cube solver was partially built in reference to the LEGO Mindstorms EV3 Education Edition that existed for a long period of time and utilized by various Rubik's cube passionate [1]. The team had made changes to the design accordingly to coincide with the algorithm that was developed and the cube size. Key elements of the design include:

### 4.1.1 The Chassis Design

The chassis of the robot was designed with two primary considerations: compactness and stability. Given that the robot's primary task is solving a Rubik's Cube—a relatively small object—the overall size of the robot does not need to be large. Therefore, the chassis was designed to be small and efficient while ensuring adequate support for all robot components.

Stability was a critical requirement due to the precision needed during the cube-solving process. The chassis had to provide a firm and balanced foundation to support the robot's mechanisms, including the motorized arm, rotating base, and sensors. A stable chassis minimizes vibrations and unwanted movement, which could interfere with the robot's performance.

To meet these requirements, the chassis was constructed using LEGO EV3 structural components, ensuring lightweight yet rigid support. The design incorporated a wide base to distribute the robot's weight evenly, reducing the risk of tipping during operations. Additional reinforcements were added to maintain structural integrity under dynamic conditions, such as motor-induced forces during cube manipulation.

The initial design of the chassis is illustrated in **Figure 1**, showcasing its compact size and the arrangement of key support elements. Further iterations of the chassis were tested to optimize stability and accommodate the other mechanical components.

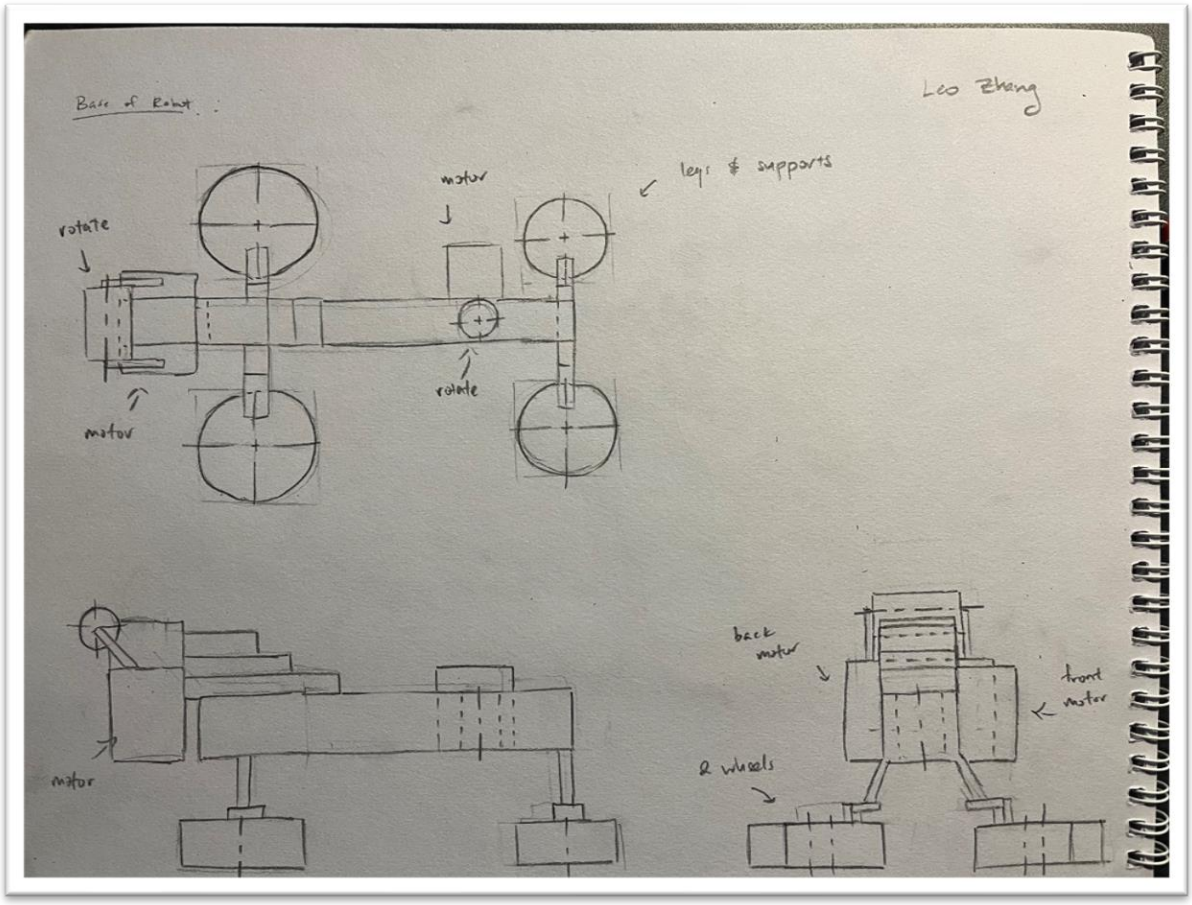


*Figure 1: A sketch of the chassis*

### 4.1.2 Gripping Arm

The robot's gripping arm is specially designed to hold the cube securely during the solving process. It adjusts its position to grip the cube tightly, preventing it from slipping while allowing precise rotations **(Figure 2)**. The arm is also able to flip the cube when needed, so all faces are exposed for scanning and manipulation. This feature is crucial for keeping everything aligned as the cube transitions between moves, ensuring the robot can access every face accurately.

The gripping arm is powered by a motor at the back, which uses a motor encoder to move with pinpoint precision. The encoder helps the arm reach the exact position it needs to grip, release, or flip the cube without making mistakes. This level of accuracy is required to reduce the chance of errors and keep the process running smoothly.

The arm works in three main states:

1. **Default Position (Figure 3):** The arm stays in a neutral position, away from the cube, so the base can rotate freely or prepare for the next step.
2. **Gripping State (Figure 2):** The arm moves down to grip the cube firmly, holding it steady for rotations or flips.
3. **Releasing State (Figure 4):** The arm lifts and releases the cube, allowing it to move to a new position for further manipulation. Then it will return to the default position, which pushes the cube back to its original position.

These states are carefully synchronized with the robot's movements, so the arm only engages or releases the cube at the right time. This design improves the reliability of the gripping mechanism and makes the entire solving process more accurate and efficient.

*Figure 2: Gripping State*

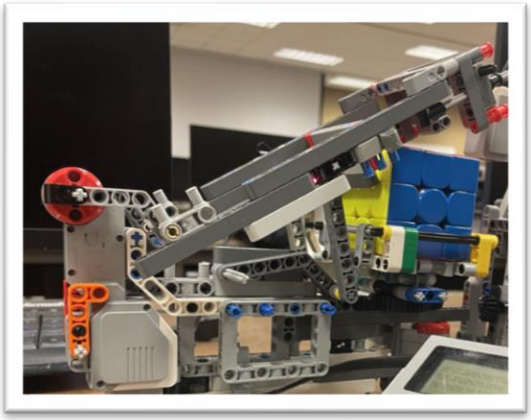

*Figure 3: Default State*

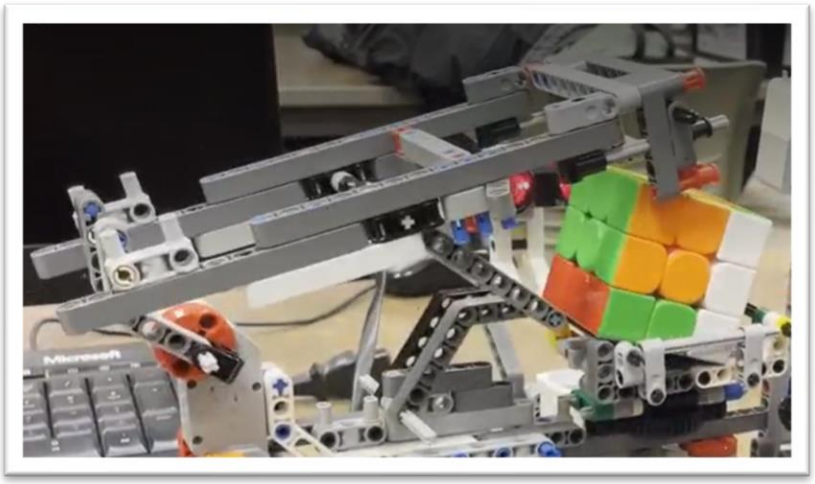

*Figure 4: Release State*

### 4.1.3 Rotating Base

The robot's motorized turntable forms the base of the system, allowing it to rotate the cube. A motor connected to a gear mechanism drives the turntable, converting the motor's movement into precise rotations of the base **(Figure 5)**. The gear setup allows the base to turn in steps of 45° and 90°, making it easy to align and manipulate specific layers of the cube. These controlled movements are essential for the solving algorithm to perform its steps correctly.

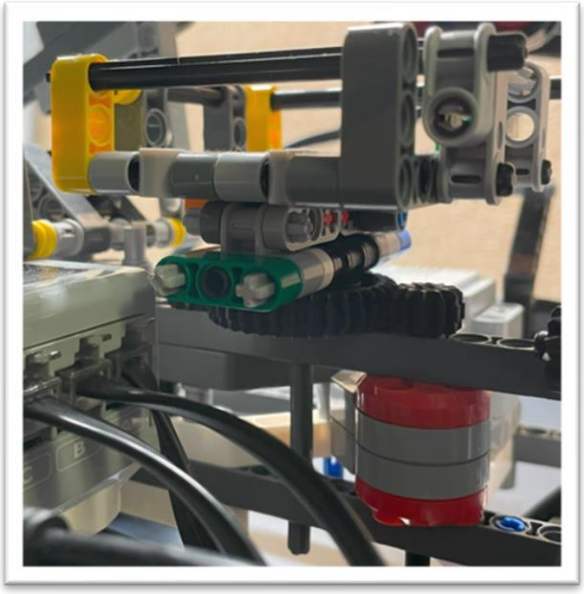

*Figure 5: Rotating Mechanism*

### 4.1.4 Color Detection:

The EV3 Color Sensor is a key part of our Rubik's Cube solver, responsible for reading the cube's color configuration. A motor moves the sensor forward and backward **(Figure 6),** using a gear system: a horizontal gear connected to the motor drives a vertical gear **(Figure 7).** This setup converts the motor's rotation into precise forward and backward motion, allowing the sensor to scan the cube faces.

The sensor works by scanning one face of the cube at a time, detecting the colors on all its squares. To scan all six faces, the gripping arm and rotating base work together to reposition the cube, ensuring everything is lined up perfectly.

Once all the faces are scanned, the robot creates a digital map of the cube's initial state. This map is then passed to the solving algorithm, which calculates the sequence of moves needed to solve this puzzle. The accuracy of this process is crucial because even a tiny error in detecting the colors can lead to an incorrect solution. That said, the EV3 Color Sensor isn't perfect. It sometimes struggles to identify certain colors, which can throw off the entire scanning process. If a color is misread, the cube's digital representation ends up wrong, and the solving algorithm fails. These challenges highlight the importance of precise and reliable state detection.

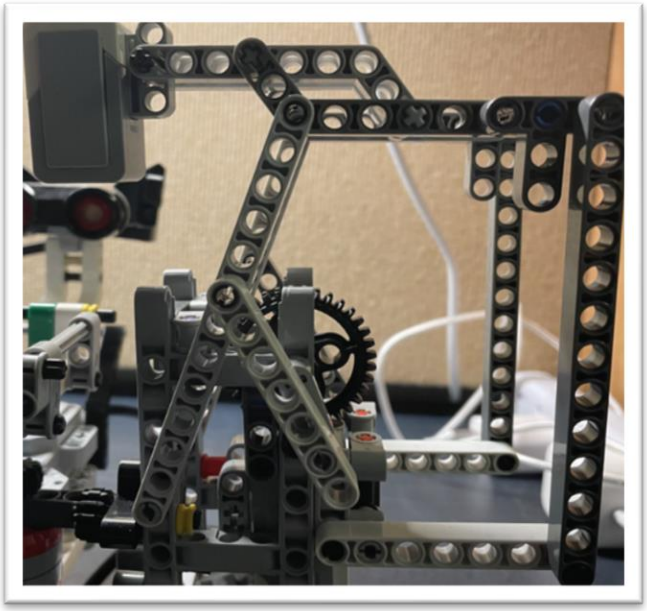

*Figure 6: Detecting Colour Mechanisms*

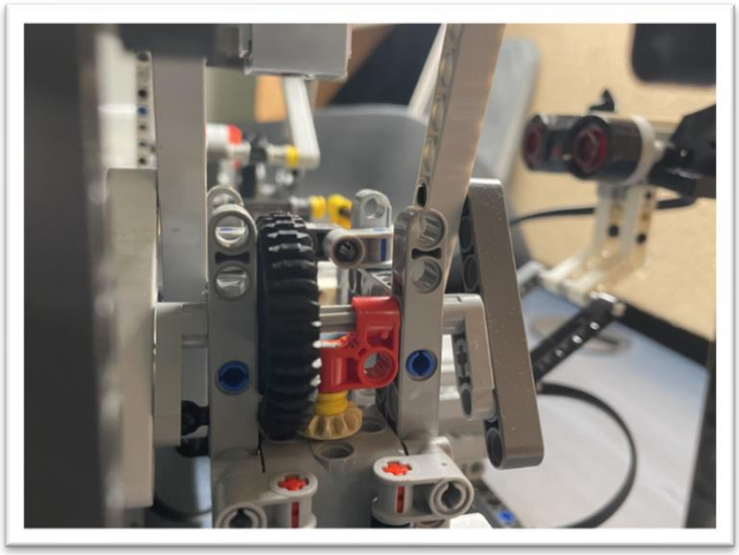

*Figure 7: Gear Mechanics*

### 4.1.5 Ultrasonic Sensor

An ultrasonic sensor **(Figure 8)** is used to ensure the cube is properly placed before scanning begins. If the cube is not positioned correctly, the system will stop, preventing inaccurate scans and ensuring reliable operation. This feature is critical for avoiding errors caused by misaligned cubes.

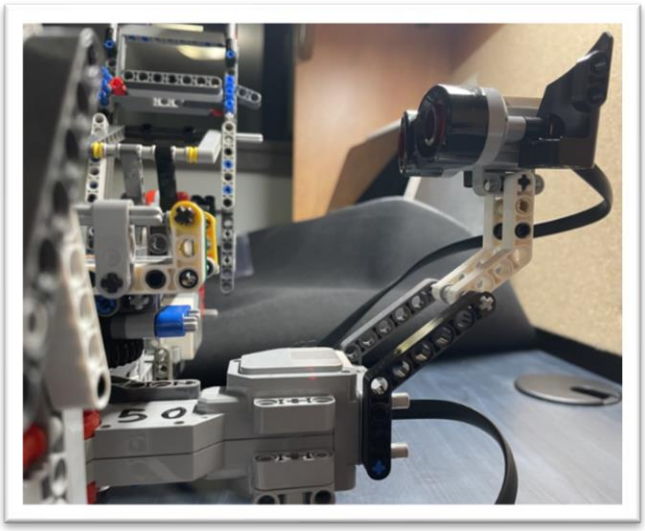

*Figure 8: Ultrasonic Sensor*

## 4.2 Design decisions and tradeoffs

### 4.2.1 The Base

During the early stages of designing the Rubik's Cube solver, the team started with a larger base. However, a problem quickly arose: the base was off-center, which caused the color sensor to misalign with the cube. This misalignment led to inaccurate color readings, making it difficult for the system to detect the cube's state properly. On top of that, when the cube was being moved or flipped, it shifted beyond what was expected, further throwing off the sensor and causing a series of errors.

To fix this, the team has decided to make the base smaller. This change helped keep the cube in a fixed position throughout the process. With the cube stabilized, the color sensor could stay properly aligned, leading to more accurate reading. As a result, the scanning became much more reliable, and the solving algorithm could work as intended. While the smaller base meant the robot was less stable overall, this trade-off was worth it for improved accuracy and functionality.

### 4.2.2 The Arm

The second problem faced during the design process was with the griping arm of the Rubik's Cube solver. Initially, the arm was designed with a smaller grip to hold the cube in place, but this proved to be inadequate. The smaller grip couldn't keep the cube stable, causing it to slip during rotations and making it hard to align properly between moves. To fix this, the arm was redesigned with a larger grip **(Figure 9)** to ensure the cube stayed securely in place throughout the solving process. This change was crucial for preventing slippage and maintaining the accuracy of the solving algorithm.

However, the redesign wasn't without its challenges:

1. **Oversized grip pieces**: While testing larger pieces to improve stability, it was found that some grips were too big. They got in the way of the base's rotation, blocking the motorized turntable and making it hard for the robot to rotate the cube in the precise 45° or 90° steps needed for solving. This mechanical issue was a major obstacle since the solving algorithm depends on exact rotational movements.
2. **Not enough friction**: Even with a larger grip, another problem arose—there wasn't enough friction to hold the cube firmly in place. During rotations or flips, the cube experienced significant forces, and the grip couldn't handle it. The cube would slip or shift out of position, which made the robot's performance inconsistent and unreliable.

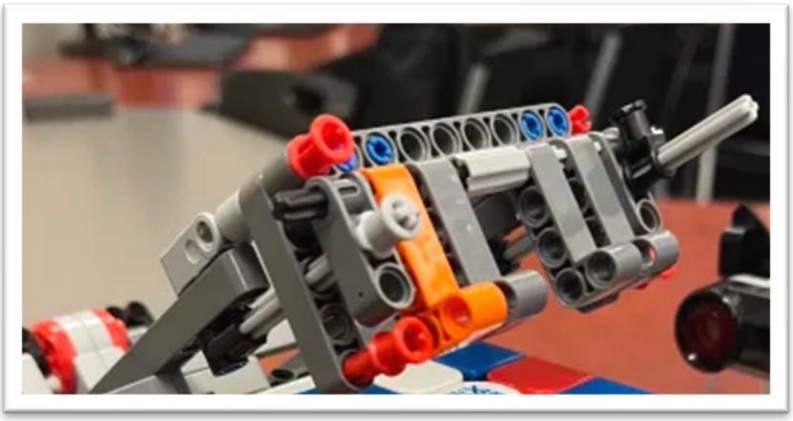

*Figure 9: Improved Gripping Mechanism*

Another issue arose with the arm's pushing mechanism. When the arm was lowered to push the cube back into the base, it didn't apply enough force, leaving the cube misaligned. To solve this, the arm was redesigned so the pushing part **(Figure 10)** fit the cube better and applied just the right amount of pressure. This adjustment ensured the cube stayed in place during rotations and flips, making the entire system more reliable.

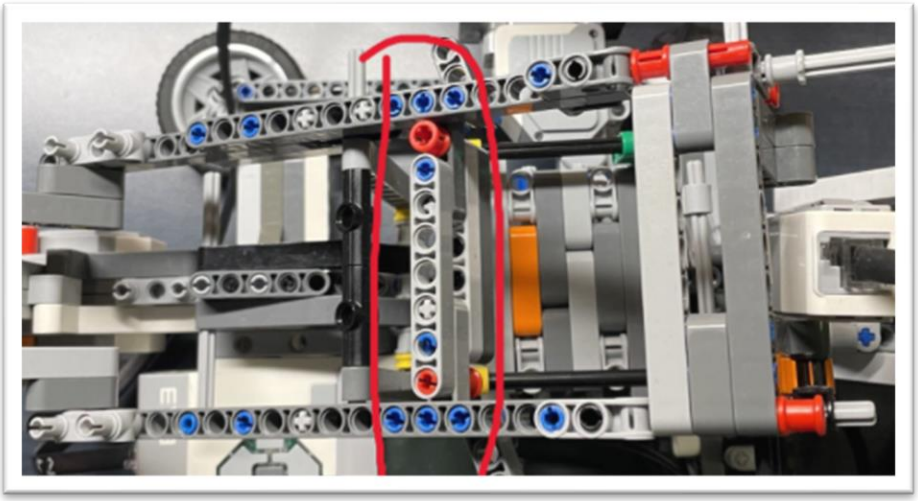

*Figure 10: Push Mechanism*

# 5.0 Software Design and Implementation

## 5.1 Introduction

The main goal of the software is to control the robot's hardware, detect the state of the Rubik's Cube, and execute an algorithm to solve it. The software works with robots' motors, sensors, and actuators. It first scrambles a solved cube and stores the scramble in a text file. The software then processes the scramble, finds the solution, and controls the robot to solve the cube. The cube is always inserted in a fixed orientation (white on top, green in front), and the software tracks its orientation as the robot performs the solve.

## 5.2 Software Design Overview

The program was broken into logical blocks to simplify development and debugging:

1. **Sensor Initialization and Validation**:
    i. Functions like "ultrasonic" ensure the cube is placed correctly before any operation begins.
    ii. These tasks are critical for starting the robot safely and ensuring accurate sensor readings.
2. **Cube Manipulation:**
    i. Functions such as "flip", "base", and "turnWhileHold" control motorized movements to orient and rotate the cube.
    ii. Dividing these tasks ensures modular control over specific movements.
3. **Layer Rotation:**
    i. The "turnLayer" function combines cube orientation and rotation logic, centralizing operations for individual layers.
4. **Color Detection and Data Storage:**
    i. The "colourSensor" function captures color data from cube faces, storing it in the "colour" array for solving algorithms.
5. **File Handling:**
    i. Functions like "randomScramble" and "readAndTurn" handle input/output to read scramble instructions or save scramble sequences.
6. **User Interaction:**
    i. The "main" task manages user input and determines program flow based on button presses.

**Justification:** These tasks were chosen to ensure modularity, simplify debugging, and facilitate future expansions (e.g., integrating advanced solving algorithms).

## 5.3 Demo Task List

Originally, the task list included major tasks such as solving or scrambling the cube. However, the team realized that the tasks provided were hard to explicitly grade since they involved many smaller components. Therefore, the new task list divided the main tasks into subtasks, allowing for easier test runs and evaluation.

**Demo Tasks:**

- Robot is set to the initial position manually (robot arm is at rest, base is at 90 degrees, colour sensor is at the very back)

- After starting the program, a set of instructions is displayed on the robot display panel: The user can choose from pressing the Left or Right buttons to scramble the cube or pressing the Enter button to colour sense and then solve the cube. If no button is pressed, nothing happens.

- Ultrasonic sensor must sense the existence of a cube to start any actions. No action is performed if the cube is not in place even after buttons are pressed. Instructions are displayed on the display panel.

- After the cube is placed and either the Left or Right button is pressed, the robot will start scrambling the cube to a randomized pattern.

- By pressing the Enter button, the robot will colour sense all faces of the cube. A text line will appear on the display panel once the robot finishes colour sensing.

- Using the algorithm in Python and reading the current state of the cube, a solution to solve the cube will be provided (not marked). The robot will read the solution to solve the cube.

- By pressing the Enter button again, the robot should begin to solve the cube. Robot will complete a series of moves (flips, rotating the base and layers) until the cube is fully solved.

- The time it takes the robot to solve the cube is displayed on the console.

- The user can press the Down button to end the program.

## 5.4 Function Descriptions

- **ultrasonic**
    a. Parameters: None.
    b. Return Type: Void.
    c. Description: Ensures the cube is placed in the base before proceeding. It continuously checks the ultrasonic sensor (SensorValue[S2]) until the cube is detected within the correct range. It provides visual cues on the robot display panel to place the cube in the base.
    d. Author: Siyuan

- **Flip**
    a. Parameters: int flipTimes, bool updateFace.
    b. Return Type: Void.
    c. Description: Rotates the cube around its axis by controlling motor motorA. It updates the face orientation array faces if updateFace is true.
    d. Author: Aston

- **base**
    a. Parameters: int direction, int degree, bool updateFace.
    b. Return Type: Void.
    c. Description: Rotates the base of the cube either clockwise or counterclockwise by a specified angle (degree). Updates faces for the new orientation.
    d. Author: Leo

- **turnWhileHold**
    a. Parameters: int direction, int degree.
    b. Return Type: Void.
    c. Description: Turns only the bottom face of the cube by holding the cube with the arm and rotating the base. Rotates the base by calling the “base” function.
    d. Author: Aston

- **turnLayer**
    a. Parameters: char layer, int direction.
    b. Return Type: Void.
    c. Description: Reorients the cube so the specified layer is at the bottom and performs the rotation using turnWhileHold.
    d. Author: Aston

- **colourSensor**
    a. Parameters: None.
    b. Return Type: Void.
    c. Description: Reads and stores the color of each square per face on the cube using the color sensor (SensorValue[S1]). Data is stored in the colour 2D array. (This function is not used because the colour sensor did not correctly detect the colours of each face.)
    d. Author: Siyuan

- **determineFace**
    a. Parameters: char originalFace.
    b. Return Type: Char.
    c. Description: Returns the current face that corresponds to the face when the cube was in the original position ("originalFace").
    d. Author: Leo

- **randomScramble**
    a. Parameters: int numMoves.
    b. Return Type: Void.
    c. Description: Randomly generates a number of moves based on the input "numMoves" and outputs it to a text file.
    d. Author: Leo

- **readAndTurn**
    a. Parameters: int fileChoice.
    b. Return Type: Void.
    c. Description: Reads an input file based on "fileChoice" which contains moves for turning the cube. Calls corresponding functions for the robot to move accordingly. This function is used to move the robot for both scrambling and solving the cube.
    Author: Cean
- **WriteToFile**
    a. Parameters: None.Return
    b. Type: Void.
    c. Description: outputs a string representation of the current state of the cube to a text file(this function was not used since the colour sensour did not work)
    d. Author: Cean

- **Main**
    a. Parameters: None.
    b. Return Type: Void.
    c. Description: Configures the robot, including sensors and buttons. Executes the program by calling upon various functions to move the robot. Improves user experience and clarity by outputting instructions on the robot.
    d. Author: Siyuan

## 5.5 Data Storage in the Program

- **Arrays**:
    - colour[6][9] : Stores the detected colors for each face, with indices corresponding to individual squares on each face.
    - faces[6]: Represents the current orientation of cube faces. Contains characters of 'U','D','F','B','L','R' corresponding to which face is turned (up, down, front, back, left, right) for each move.
- **Files**:
    - Scramble sequences and solving steps are saved in external files (scramble.txt, output.txt) for persistence and debugging.

**Rationale**: Arrays provide efficient in-memory storage for real-time operations, while files enable reproducibility and analysis.

## 5.6 Software Design Decisions

- **File I/O:**
    - Trade-off: Using files for input/output increases reproducibility but adds overhead during runtime.
- **Motor Encoder Feedback:**
    - The decision to use motor encoders for precise control reduced mechanical error but required additional tuning for speed and accuracy.
- **Modularity:**
    - Breaking down tasks into functions like base and flip ensured reusability but increased inter-function dependency, requiring careful integration testing.

Two Algorithms were Implemented in this project:

**Algorithm 1: IDA* Search (Iterative Deepening A* Search) [3]**

- Written in python, easy to implement and understand, also has many
- Trade-off: python runs slower than other languages like C++ or C, and it takes up more memory.
- Input: A string representation of the scrambled cube's state.

- IDA* is efficient and fast compared to other algorithms, it is considered the best algorithm (that is not customized specifically for cubing) to use to solve a Rubik's cube.
- For short solutions ranging from 1 to 7, the algorithm runs extremely fast, outputting the solutions within 0.5 to 2 seconds
- However, the trade-off is its time complexity grows rapidly as the number of moves increases. For scrambles involving 8 or more moves, it would take more than 5 minutes to run the program.
- We ended up not using this algorithm, as it takes a string representation of the cube as its input. This requires us to use the colour sensor, however, since the EV3 colour sensor was not able to scan all colours of the cube. Additionally, the slow run time of the IDA* algorithm for larger scrambles made it impractical for solving more complex cubes, further contributing to our decision to explore alternative methods.

**Algorithm 2: Selenium Web Solution**

- A python code written by Cean Liu to retrieve the solution from Online Rubik's Cube Solver [2] using python selenium [4].
- Online Rubik's Cube Solver [3] uses an algorithm designed specifically to cubing, which means the time to retrieve the solution remains relatively constant (around 1 minute). The moves it takes to solve the cube is slightly higher than Algorithm 1, with an average of 20 moves.
- If the scramble move is higher than 8, this algorithm is great for solving the cube as it would take significantly less time to calculate the moves than Algorithm 1, however, the trade-off is the moves is slightly longer, and it would take a longer time for 7 moves or lower.
- Because using Selenium to web scrape data off of a website involves with the use of Internet, network connection issues could potentially slow things down or even crash the program.
- We still ended up adopting this approach due to the consistent time to retrieve the solution. And the input method, which only requires a sequence of scramble moves, making it easier to implement and work with.

## 5.7 Testing and Validation

*Table 2 2: Test Cases for Software Implementation*

| Test Case | Function Tested | Expected Outcome | Result |
|---|---|---|---|
| Sensor Value Range | ultrasonic | Displays warning if cube not detected, robot does not start until cube is detected | Pass |
| Face Rotation | flip and base | Cube rotates as expected | Pass |
| Color Detection | colourSensor | Accurate color readings stored | Fail (alternative solution was implemented) |
| Layer Turning | turnLayer | Correct layer rotates | Pass |
| Scramble Generation | randomScramble | Generates valid scramble sequences | Pass |
| Cube Orientation | determineFace | Returns correct original face | Pass |
| Solving Pattern | Python files (implemented) | Solves the cube when the pattern of moves is followed | Pass |
| Button Progamming | main | Correct steps are followed (scramble/solve) depending on which button is pressed | Pass |
| Full System Test | All functions (integration) | Cube scrambled and solved correctly | Pass |

## 5.8 Problem Resolution

1. **Problem**: Motor alignment inconsistencies during rotations.
    a. **Resolution**: Implemented motor encoder-based feedback and fine-tuned speed and delays.
2. **Problem**: Sensor misalignment in “colourSensor”.
    a. **Resolution**: Added calibration routines and adjusted sensor positions for accurate readings.
3. **Problem**: Incorrect face updates during rotations.
    a. **Resolution**: Debugged “flip”, “base”, “turnLayer” and “determineFace” update logic, ensuring face orientation matches physical rotation.
4. **Problem**: Colour readings are not accurate from “colourSensor”

a. **Resolution**: Adopted alternative solution which allows system to function without the colour sensor. After randomly scrambling the cube and storing the moves in a text file, it can be inputted into the solving algorithm (implemented python file) to find a pattern of moves to solve the cube. These moves are then stored in another text file to be input back into the robot, allowing the robot to read the text file and turn the cube.

# 6.0 Verification

*Table 3: 3 Verification for Various Constraints*

| **Constraints** | **Was it met? (Yes/No)** | **How/Why? Next Steps?** |
|---|---|---|
| Meeting Safety standards | Yes | The robot will not run if:<br>1. The cube is not placed on the base<br>2. If the user did not press the button indicating the desired move they wish the perform.<br>This prevents unexpected movements and minimizes the risk of accidental operation. Reducing potential hazards and enhancing overall operational saftey./ |
| Substantial mechanical redesign of the EV3 | Yes | We changed the entirety of the EV3, which includes rotating mechanisms and custom arms to interact with the Rubik's Cube. This allowed the robot to manipulate the cube on a stable platform, ensuring precise and controlled movements during the solving process. The custom design provided better accuracy and stability for the cube's orientation, allowing the robot to solve it efficiently without compromising the structure or functionality. |
| Input Diversity | No/Yes | No: The Colour Sensor on EV3 does not functional properly, we adopted another solution using scrambling algorithms to get the initial state of the cube, bypassing the need for accurate colour sensing.<br><br>Yes: The robot has enough input diversities to work with:Ultrasonic SSensor, Motorncoder, EV3 Buttons and File/IO are all valid inputs for the project. |
| Use of timers | Yes | At the end of our program, the robot outputs the time taken to solve the cube onto the EV3 screen |
| Software Requirements | Yes | The program uses loops to read the solution file, then uses decisions to check for what face tor turn on the cube. |

# 7.0 Project Plans

The project plan was structured to ensure an equitable division of responsibilities while promoting collaboration and flexibility among team members. The team consisted of four members, each contributing to different aspects of the project.

- **Mechanical Design:** Two team members were primarily responsible for the mechanical aspects of the project. Their tasks included designing, constructing, and testing the robot's physical components, such as the chassis, motorized arm, and rotating base.

- **Software Development:** The other two members focused on the software aspects, including programming the robot's functionality, integrating sensors, and ensuring seamless interaction between hardware and software components.

Despite the division of responsibilities, the team adopted a collaborative approach to ensure smooth progress and comprehensive testing:

- Regular team discussions were held to share ideas, review progress, and refine designs based on testing outcomes.

- Team members provided mutual assistance when challenges arose or when individual tasks were completed ahead of schedule. This ensured that delays in one area did not hinder overall project progress.

The final project presentation was a collective effort, with all team members contributing to the preparation and delivery of the slideshow. This ensured that the presentation accurately reflected the contributions and insights of the entire team.

The schedule of completing each milestone of the project was also planned. It is illustrated in **Figure 11.**

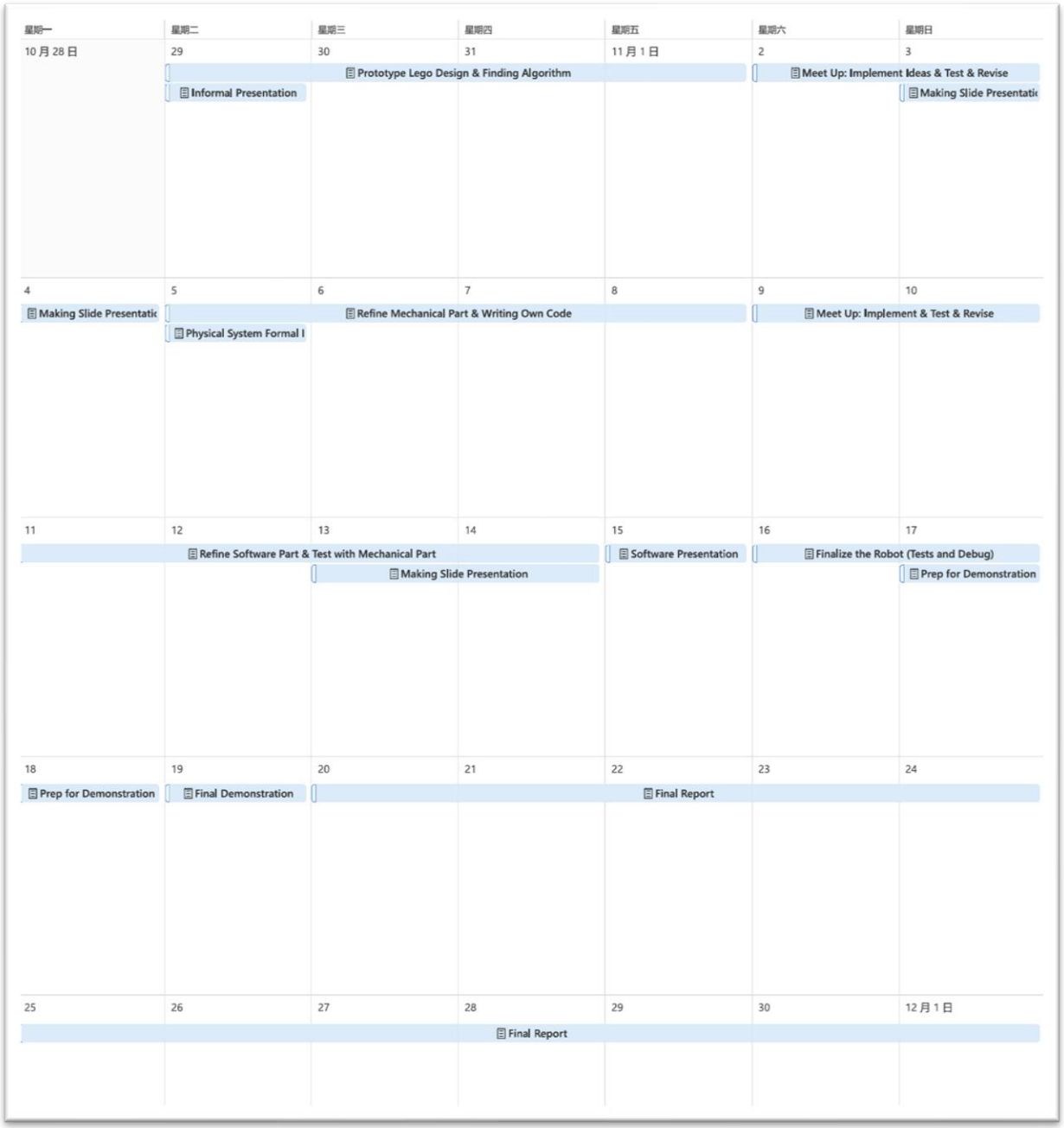


*Figure 11: Schedule of Completion*

# 8.0 Conclusion

The objective of this project was to design and implement a robotic system capable of solving a Rubik's Cube using LEGO Mindstorms EV3 components. The mechanical design, inspired by existing models, was modified to accommodate precise cube manipulation and ensure compatibility with the solving algorithm. Key mechanical features, such as the gripping arm, rotating base, and various sensors, were integrated to enhance the robot's accuracy and reliability.

On the software side, modular functions were developed to manage motorized movements, sensor readings, and file handling. Two solving algorithms were explored: IDA* for theoretical efficiency and a Selenium-based web solution for consistent performance. Due to hardware limitations with the EV3 color sensor, the team adopted a practical workaround involving scramble storage and file-based solutions, ensuring the system could operate effectively without direct color detection.

Through iterative testing and design adjustments, the robot successfully achieved its primary goal of scrambling and solving a Rubik's Cube, meeting safety and operational standards. Despite challenges with sensor accuracy and mechanical alignment, the team implemented effective solutions, resulting in a functional and reliable system. This project demonstrates the successful integration of mechanical, software, and algorithmic components to achieve an automated Rubik's Cube-solving robot.

# 9.0 Next Step/Recommendation

To guide future teams continuing this project, several areas of improvement and development are recommended:

1. **Sensor Accuracy Enhancement:**
    - **a.** Consider upgrading the color sensor to a more advanced model or exploring alternative vision systems to increase accuracy in color detection. Machine vision solutions could be integrated to eliminate reliance on scramble storage methods.

2. **Mechanical Stability:**
    - a. Refining the gripping mechanism and rotating base to improve precision and reduce misalignments will enhance the robot's reliability. Introducing additional structural reinforcements or adjusting torque control in motors could also mitigate any mechanical stress issues.

3. **Algorithm Optimization:**
    **a.** Further exploration of advanced solving algorithms like Kociemba's Two-Phase Algorithm could increase efficiency and reduce solve time. Alternatively, refining the existing IDA* implementation to run on embedded platforms without performance degradation would also be beneficial.

4. **User Interface and Automation:**
    **a.** Developing a more intuitive user interface with real-time status updates and visual feedback would improve the overall user experience. Automation of the entire solving process, from cube scanning to solution execution, could streamline the workflow.

5. **Hardware Limitations and Expansion:**
    a. Future iterations could explore migrating the software onto a more powerful microcontroller or integrating cloud-based computation for complex algorithms, addressing the limitations of LEGO Mindstorms EV3 hardware.

# References

[1] Mindcuber, https://mindcuber.com/mindcub3r/MindCub3r-v1p0.pdf (accessed , 2024).

[2] M. Beke, “Online Rubik’s Cube Solver,” *Rubik’s Cube Solver*. https://rubiks-cube-solver.com/.

[3] FarhanShoukat, “GitHub - FarhanShoukat/Rubiks-Cube-Solver: a Comprehensive Comparision of IDFS and IDA* in Solving Rubik’s Cube,” *GitHub*, 2018. https://github.com/FarhanShoukat/Rubiks-Cube-Solver

[4] Selenium, "Selenium WebDriver," [Online]. Available: https://www.selenium.dev. Accessed: Nov. 19, 2024].

# Appendix

## The Code Source:

### “RunRobot"

```
#include "PC_FileIO.c"


int colour [6][9];

char faces [6] = {'U','F','R','B','L','D'}; //start with original move orientation


void ultrasonic ()

{

while (SensorValue[S2] <= 2 || SensorValue[S2] >= 9)

{

displayTextLine(14,"Cube must be placed in base");

displayTextLine(15,"for program to run");

}

wait1Msec(2000);

}


void flip(int flipTimes, bool updateFace)

{

const int POS1ANGLE = 130, POS2ANGLE = 195;

for (int i = 0; i < flipTimes; i++)

{

nMotorEncoder(motorA)=0;

//phase 1

motor[motorA] = 25;

while(abs(nMotorEncoder[motorA]) < POS1ANGLE)

{}
```

```
motor[motorA] = 0;
wait1Msec(500);
//phase 2
motor[motorA] = 25;
while(abs(nMotorEncoder[motorA]) < POS2ANGLE)
{}
motor[motorA] = 0;
wait1Msec(500);
motor[motorA] = -25;
nMotorEncoder(motorA) = 0;
while(abs(nMotorEncoder[motorA]) < POS2ANGLE-POS1ANGLE)
{}
motor[motorA] = 0;
//wait1Msec(1000);
motor[motorA] = -25;
while(abs(nMotorEncoder[motorA]) < POS2ANGLE)
{}
motor[motorA] = 0;

if(updateFace)
{
//UPDATE CUBE FACES ARRAY
char temp = faces[0];
faces[0] = faces[2];
// face 1 is the same
faces[2] = faces[5];
//face 3 is same
char temp2 = faces[4];
faces[4] = temp;
```

```
faces[5] = temp2;
}
}
}

void base(int direction, int degree, bool updateFace) // CW: direction = 0, CCW: direction = 1
{
//270 for 90degree turn
nMotorEncoder(motorB) = 0;
if(direction == 0)
{
motor[motorB] = 25; // positive is CW
}
else
{
motor[motorB] = -25; // negative is CCW
}
while(abs(nMotorEncoder[motorB]) < degree)
{}
motor[motorB] = 0;
wait1Msec(500);

if(updateFace && degree%270 == 0){
//UPDATE CUBE FACES ARRAY
char temp [6];
for(int i=0;i<6;i++)
{
temp[i] = faces[i];
}
```

```
int numTurns = degree/270; //either 1 or 2
if(direction == 1 && numTurns == 1)
{
numTurns = -numTurns;
}
int newFace = 0;
for(int i=1; i<5; i++)
{
newFace = i+numTurns;
if(newFace>4)
{
newFace -= 4;
}
else if(newFace<1)
{
newFace += 4;
}
faces[i] = temp[newFace];
}
}
wait1Msec(200);
}

void turnWhileHold(int direction, int degree)
{
//arm holds cube
const int POS1ANGLE = 130;
nMotorEncoder(motorA)=0;
motor[motorA] = 25;
```

```
while(abs(nMotorEncoder[motorA]) < POS1ANGLE)
{}
motor[motorA] = 0;
//turn base
base(direction, degree, false);
//remove arm
nMotorEncoder(motorA)=0;
motor[motorA] = -25;
while(abs(nMotorEncoder[motorA]) < POS1ANGLE)
{}
motor[motorA] = 0;
}

void turnLayer(char layer, int direction) // front layer is the side with no obstructions, up is facing up
{
//GET SPECIFIED LAYER TO THE BOTTOM
if(layer == 'F')//front face
{
//turn cube CW by 90 and flip once
base(0,270,true);
flip(1,true);
}
else if(layer == 'B')//back face
{
//turn cube CCW by 90 and flip once
base(1,270,true);
flip(1,true);
}
else if(layer == 'L')//left face
```

```
{
//flip once
flip(1,true);
}
else if(layer == 'R')//right face
{
//turn cube CW by 180 and flip once
base(0,540,true);
flip(1,true);
}
else if(layer == 'U')//up face
{
//flip twice
flip(2,true);
}
//nothing required for down face

//TURN THE SPECIFIED LAYER
const int TURN_DEGREE = 270;
if(direction == 0)//turn CW
{
turnWhileHold(1,TURN_DEGREE);
}
else if (direction == 1)//turn CCW
{
turnWhileHold(0,TURN_DEGREE);
}
}
```

```
void colourSensor()
{
const int CENTREANGLE = 440, SIDEANGLE = 360;
for (int i=0; i<6; i++)
{
nMotorEncoder[motorC] = 0;
motor[motorC] = -80; //moving forward
while (abs(nMotorEncoder[motorC])< CENTREANGLE)
{}
motor[motorC] = 0;
colour[i][0] = SensorValue[S1]; //stores centre colour
wait1Msec(1000);

nMotorEncoder[motorC] = 0;
motor[motorC] = 80; //moving bacward
while (abs(nMotorEncoder[motorC])< 90)
{}
motor[motorC] = 0;

for (int j=1; j<8; j++)
{
colour[i][1+j] = SensorValue[S1];
wait1Msec(200);
base(1,135,false);
// minor position changes to the sensor for accurate readings
if(j == 3)
{
nMotorEncoder[motorC] = 0;
motor[motorC] = 80; //moving backward
```

```
while (abs(nMotorEncoder[motorC])< 20)
{}
motor[motorC] = 0;
}
else if(j == 4)
{
nMotorEncoder[motorC] = 0;
motor[motorC] = -80; //moving forawrd
while (abs(nMotorEncoder[motorC])< 20)
{}
motor[motorC] = 0;
}
else if(j == 5)
{
nMotorEncoder[motorC] = 0;
motor[motorC] = 80; //moving bcakward
while (abs(nMotorEncoder[motorC])< 20)
{}
motor[motorC] = 0;
}
else if(j == 6)
{
nMotorEncoder[motorC] = 0;
motor[motorC] = -80; //moving forward
while (abs(nMotorEncoder[motorC])< 20)
{}
motor[motorC] = 0;
}
else if(j == 7)
```

```
{
nMotorEncoder[motorC] = 0;
motor[motorC] = 80; //moving bcakward
while (abs(nMotorEncoder[motorC]) < 40)
{}
motor[motorC] = 0;
}
wait1Msec(500);

}
nMotorEncoder[motorC] = 0;
motor[motorC] = 80; //moving backward
while (abs(nMotorEncoder[motorC]) < 310)
{}
motor[motorC] = 0;
base(1,135,false);
flip(1,false);
if (i==3)
{
base (0, 270, false);
flip (1,false);
}
if (i==4)
{
flip(1,false);
}
}
base(1,270,false);
}
```

```
char determineFace(char originalFace) //original face is the face that the algorithm wants to turn
{
int index = -1;
for(int i=0;i<6;i++)
{
if(originalFace == faces[i]){ // faces is the current orientation of the cube
index = i;
}
}
char newMove[6] = {'U','F','R','B','L','D'};
return newMove[index];
}

void randomScramble(int numMoves)
{
int faceNum = 0;
int direction = -1;
char face = 'N'; // Nothing

TFileHandle fout;
bool fileOkay = openWritePC(fout,"scramble.txt");

if(!fileOkay)
{
displayTextLine(5,"ERROR");
wait1Msec(5000);
}
else
```

```
{
writeLongPC(fout,numMoves);
writeEndlPC(fout);
for (int i = 0; i < numMoves; i++)
{
faceNum = (rand() % (6-1))+1;
if(faceNum == 1)
{
face = 'U';
}
else if(faceNum == 2)
{
face = 'F';
}
else if(faceNum == 3)
{
face = 'R';
}
else if(faceNum == 4)
{
face = 'B';
}
else if(faceNum == 5)
{
face = 'L';
}
else
{
face = 'D';
```

```
}
direction = (rand() % (1-0));
fileWriteChar(fout,face);
writeEndlPC(fout);
writeLongPC(fout,direction);
writeEndlPC(fout);
}
closeFilePC(fout);
}
}

void readAndTurn(int fileChoice){ //fileChoice: 0 for solve, anything else for scramble
TFileHandle fin;
bool fileOkay = false;
if(fileChoice == 0)
{
fileOkay = openReadPC(fin,"output.txt");
}
else //randomly scramble
{
randomScramble(8); //change number to change number of moves to scramble
fileOkay = openReadPC(fin,"scramble.txt");
}
clearTimer(T1);
if(!fileOkay)
{
displayTextLine(5,"error");
wait1Msec(5000);
}
```

```
else
{
int lineNum = 0;
readIntPC(fin, lineNum);
for (int line = 0; line < lineNum*2-1; line+=2){
char layer;
int dir;
readCharPC(fin, layer);
readIntPC(fin, dir);
turnLayer(determineFace(layer),dir);
}
float turnTime = time1[T1]/1000.0;
displayTextLine(1,"Time taken to solve:");
displayTextLine(2,"%d",turnTime);
wait1Msec(5000);
closeFilePC(fin);
}
}

task main()
{
//configuring all sensors
SensorType[S2] = sensorEV3_Ultrasonic;
wait1Msec(50);

SensorType[S4]= sensorEV3_Color;
wait1Msec(50);
SensorMode[S1] = modeEV3Color_Color;
wait1Msec(50);
```

```
bool end = false;
while(!getButtonPress(buttonAny))
{
displayTextLine(4,"Left or Right Buttons");
displayTextLine(5,"for Scrambling the Cube");
displayTextLine(7,"Enter Button for Colour");
displayTextLine(8,"Sensing and Solving the Cube");
displayTextLine(10,"Down Button for Exiting");
displayTextLine(11,"the Program");
wait1Msec(10);
}

while (!end)
  {

    // Check for left or right button press to scramble
    if (getButtonPress(buttonLeft) || getButtonPress(buttonRight))
    {
        ultrasonic();
      readAndTurn(1);  // Scramble
      wait1Msec(500);  // Delay after action to avoid multiple triggers too quickly
    }
    // Check for Enter button press to color sense and solve
    else if (getButtonPress(buttonEnter))
    {
        ultrasonic();
```

```
        //colourSensor();
        displayTextLine(4,"Colour Sensing is");
        displayTextLine(5,"finished, press the");
        displayTextLine(6,"Enter Button again");
        displayTextLine(7,"to solve");
        displayTextLine(8," ");
        while(!getButtonPress(buttonEnter))
        {}
                ultrasonic();
        readAndTurn(0);  // Color sense then solve
        wait1Msec(500);  // Delay after action to avoid multiple triggers too quickly
      }
      // Check for Down button press to end the program
      else if (getButtonPress(buttonDown))
      {
        end = true;  // Ends the program
      }

      wait1Msec(50);  // Small delay to prevent excessive CPU usage
    }
}
```

“readandwritefile"

```
#include "PC_FileIO.c"

// Global array
string colour1[6][9];
const int FACE_NUM = 6;
const int TILE_NUM = 9;



void readAndTurn(){
TFileHandle fin;
bool fileOkay = openReadPC(fin,"output.txt");
if(!fileOkay)
{
displayTextLine(5,"error");
wait1Msec(5000);
}
else
{
int lineNum = 0;
readIntPC(fin, lineNum);
displayTextLine(3,"%f",lineNum);
for (int line = 0; line < lineNum; line++){
string s;
readTextPC(fin, s);
displayString(line,s);
}
wait1Msec(5000);
closeFilePC(fin);
```

```
}
}


void writeToFile() {  // No parameters, just use global array
   TFileHandle fout;
   openWritePC(fout, "input.txt");

   int faces[6] = {0,5,1,4,3,2};
writeLongPC(fout,9);
for(int i = 0 ; i<FACE_NUM; i++)
{
for(int j = 0; j < TILE_NUM-2; j+=3)
{
string face_bar;
int index = faces[i];
stringFormat(face_bar,
"%s%s%s%s%s%s%s","[",colour1[index][j],",",colour1[index][j+1],",",colour1[index][j+2],"]\n");
writeTextPC(fout,face_bar);
}
writeEndlPC(fout);
}
   closeFilePC(fout);
}


task main() {
   // Initialize array with zeros
// {0,  0,0,0,0,0,0,0,0}
```

```
string hi[6] = {"W","R","Y","O","B","G"};
   for(int i = 0; i < 6; i++) {
      for(int j = 0; j < 9; j++) {
         colour1[i][j] = hi[i];
      }
   }

   writeToFile();  // No parameters needed
readAndTurn();
}
```